

WIP: Formalizing Fault Injection at the Microarchitectural Level

Leslie Fifanon¹, Pierre Wilke¹, Damien Couroussé², Mathieu Jan³, and
Guillaume Hiet¹

¹ CentraleSupélec, Inria, CNRS, University of Rennes, France
`firstname.lastname@centralesupelec.fr`

² Univ. Grenoble Alpes, CEA, List, F-38000, Grenoble, France
`firstname.lastname@cea.fr`

³ Univ. Paris Saclay, CEA LIST, France
`firstname.lastname@cea.fr`

Abstract. Fault injection attacks perturb a circuit at run time and can defeat security guarantees that hold under nominal execution; reasoning about such attacks, and about the countermeasures meant to thwart them, calls for formally modeling the effect of faults at the microarchitectural level. We take a first step inside Kôika, a rule-based hardware description language embedded in the Rocq prover with a formally verified compiler to Verilog. We equip Kôika with two complementary ways to model the effect of faults—a *faultable* semantics, parameterized by the injected faults and suited to deductive reasoning in Rocq, and a *saboteur-based* circuit transformation amenable to SMT-assisted verification—and formalize the *k-fault-resistant partitioning* property. As a proof of concept, we discharge this property automatically with an SMT solver on a small multiplier protected by concurrent error detection (CED). This work in progress aims at a top-down flow that compiles Kôika-based security specifications into fault-resistant Verilog.

Keywords: Fault injection · Formal verification · Hardware description language

1 Introduction

Fault injection attacks break the physical assumptions that hardware-software security contracts are built on. By glitching a circuit’s clock or supply voltage, or exposing it to electromagnetic or laser pulses, an attacker can induce faults in the microarchitecture [7], resulting in various effects at the software level such as instruction skips or data corruption. These faults can be exploited by an attacker to bypass security checks, escalate privileges, or expose secrets [9].

Current techniques for analyzing Fault-Injection Attacks (FIA). To assess the robustness of systems against fault injection attacks, approaches focus on either analyzing the consequences at the hardware level only [4,1,12] or at the software level only [11,6,5]. However, the former are unable to check if hardware

vulnerabilities are exploitable at the software level, while the latter rely on generic fault models that miss microarchitectural details [7]. Recently, formal methods have been applied to analyze the impact of fault injection attacks at the processor microarchitecture level and their consequences on software execution [14]. A new approach called k -fault-resistant partitioning was introduced to leverage redundancy-based hardware countermeasures and improve the verification process of k -fault security against FIA [15]. This technique was applied to the OpenTitan secure element to formally prove the security of its processor’s hardware countermeasure against single bit-flip injections.

Motivation for Kôika. We build on Kôika [3], a formal rule-based HDL embedded in the Rocq prover whose compilation to Verilog is formally verified, so that every circuit comes with machine-checked semantics. Kôika has already served to prove hardware security properties, namely those related to shadow stacks [2]—but only against *software* attacks. Reasoning about fault injection instead requires *enriching* the semantics of Kôika to represent the effect of faults and to reason about countermeasures. This is a *top-down* approach: from a specification carrying security guarantees down to a Verilog circuit that *preserves* fault resistance properties.

This paper takes a first step in that direction and makes the following contributions.

- we define formal fault models for Kôika circuits, and a faultable semantics parameterized by a fault model;
- we define a compiler pass for Kôika circuits that weaves the fault model inside the circuit by introducing so-called *saboteurs*;
- we formalize the k -fault-resistant partitioning property;
- we leverage an SMT solver to check the security property over a simple multiplier circuit with a concurrent error detector (CED).

The rest of the paper is organized as follows. Section 2 gives the necessary background about concurrent error detection schemes, the k -fault security property, and the Kôika HDL. Next, Section 3 gives a motivating example which will be used in the rest of the paper. Section 4 introduces fault modeling in Kôika, and Section 5 describes the proof methodology used. Finally, Section 6 concludes and gives perspectives for future work.

2 Background

2.1 Concurrent error detection (CED)

Concurrent error detection (CED) protects a functional unit by computing its characteristics via a predictor and comparing them with the unit output (Figure 1) [15]. The predictor is commonly implemented with redundant logic, and a checker mechanism verifies that both results match at runtime, as is the case for the application example in this paper (Section 3). It can also be code-based, such as an error-detection code (EDC) [13]. It is usual for the target and the predictor to be desynchronized such that the alert (A) is set only d cycles after the target output becomes available.

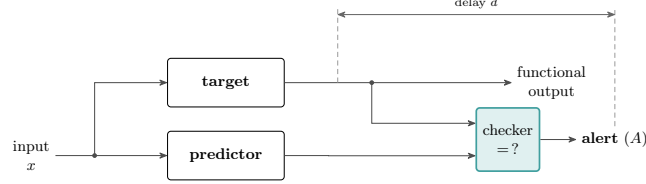


Fig. 1: Concurrent error detection (CED).

2.2 The k -fault security property

For a circuit implementing a CED with $k + 1$ redundant replicas, an intuitive security property to ensure is the k -fault security property. Formally stated, a (d, A) – CED circuit is k -fault secure if and only if, for all n , over $(n + d)$ cycles of execution, if no alert is raised, then the circuit’s output is correct for the first n cycles. Proving k -fault security requires unrolling the circuit for an indeterminate number of steps $(n + d)$ [15].

2.3 The k -fault-resistant partitioning property

In [15], Tollec et al. introduced the k -fault-resistant partitioning property as a sufficient condition to guarantee k -fault security, which requires unrolling the circuit for only a fixed number of steps $l = \max(1, d)$. k -fault-resistant partitioning is a property based on *register partitioning*, i.e., a grouping of all the circuit’s registers into disjoint sets, including those in the predictor and the checker. It requires that, in any execution step, the number of affected partitions does not grow more than the number of faults injected in the circuit unless the alert is raised (*fault confinement*) and any fault that causes immediate mismatch of outputs is detected (*output integrity*).

2.4 The Kōika language

Kōika [3] is a rule-based hardware description language (HDL) embedded in the Rocq prover (formerly Coq) as a domain-specific language, with a formally verified compiler to a synthesizable subset of Verilog. A Kōika program is a set of *registers* holding the circuit’s state, a set of *rules*, and a *schedule*. A *rule* is an atomic action that manipulates registers and describes a transition from one clock cycle to the next. Rules correspond roughly to the stages of a pipeline. The *schedule* fixes the order in which the effects of the rules *appear* to take place within a cycle. Inspired by Bluespec SystemVerilog (BSV) [10], Kōika rules execute as if they were run sequentially (*one rule at a time*, or ORAAT). Unlike BSV, which schedules rules by an opaque static analysis, Kōika makes the schedule explicit and resolves conflicts with a deterministic user-provided schedule [3]. Crucially, properties proved on a Kōika circuit carry over to its generated Verilog, thanks to the semantic preservation theorem of the Kōika compiler.

In earlier work, Baty et al. [2] reason about Kōika programs directly in Rocq. This quickly becomes too involved for large designs, and we use an SMT-assisted framework instead for proving properties.

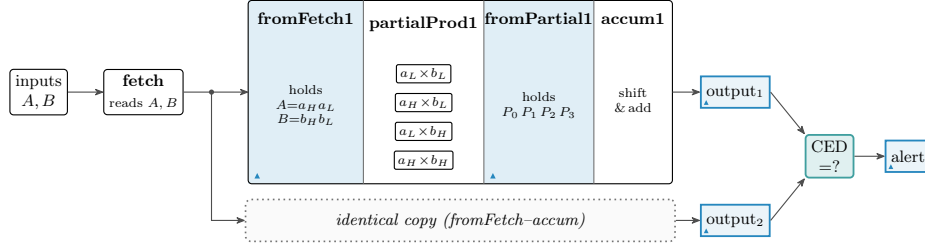


Fig. 2: A partial-product multiplier with a concurrent error-detection (CED) check.

3 Motivating example

Our running example is a pipelined partial-product multiplier (Fig. 2). A single *shared fetch* stage reads the operands A, B of size $h = 8$ bits each and feeds them to two identical pipelines. In each pipeline, *partial products* splits each operand into halves ($A = a_H a_L$, $B = b_H b_L$) and computes the four half-products $P_0 = a_L b_L$, $P_1 = a_H b_L$, $P_2 = a_L b_H$, $P_3 = a_H b_H$, and *accum1* sums them with shifts into $out = P_0 + (P_1 + P_2) \ll h + P_3 \ll 2h$; stages communicate through the pipeline registers `fromFetch1` and `fromPartial1`. The CED check compares the two pipelines' outputs, raising `alert` *immediately* on any mismatch ($d = 0$).

Because *fetch* is shared, it lies outside the duplicated region and it is excluded from the fault model (see Section 5). Both `fromFetch1` and `fromFetch2` and receive identical operands by construction; The circuit is small, letting us apply our methodology.

4 Modeling Fault Injection

4.1 Kôika semantics

A Kôika *rule* is an atomic action which computes the next values of the registers. The actions are given in Figure 3: a value v , a variable x , `skip`, a register read or write, a `let`-binding, an assignment, a conditional, a function call $f(a, \dots, a)$, a sequence, or the always-failing `abort`.

The evaluation of an action is given by the big-step judgment

$$\Gamma \vdash (l, a) \Downarrow_L^\sigma (l', v, \Gamma'),$$

where the register state σ and the schedule log L are constant throughout the execution of a given rule. Logs are data structures that accumulate register read and write events. Γ is the local variable environment that maps variables to values. The evaluation of a Kôika action a can be understood as follows:: Given

Actions $a ::= v \mid x \mid \text{skip} \mid \text{read } r \mid \text{write } r \ a \mid \text{let } x = a \text{ in } a$
 $\mid x := a \mid \text{if } a \text{ then } a \text{ else } a \mid f(a, \dots, a) \mid a; a \mid \text{abort}$
 Values $val ::= b \mid \{(k : val)^*\} \mid \{val^*\} \mid \text{enum_val}$
 Types $\tau ::= \text{Bits } n \mid \text{Struct } [(k, \tau)^*] \mid \text{Array } n \ \tau \mid \text{Enum } [variant^*]$

Fig. 3: Syntax of Kôika actions.

Γ , σ , and L , the evaluation of action a starts from a *rule log* l (the record of previous read and write events in the rule) and produces the value v , the updated rule log l' , and the updated environment Γ' . A *read* action appends $\text{rd}(r)$ to the *rule log* and returns the register's value $\sigma(r)$, while a *write* appends $\text{wr}(r, v)$; both are subject to the conflict conditions $\text{may_read}(L, r) \triangleq \text{wr}(r, *) \notin L$ (one cannot read a register if it has been written to by another rule during the current cycle) and $\text{may_write}(L, l, r) \triangleq \text{wr}(r, *) \notin (L \uparrow l)$ (one cannot write a register if it has already been written during this cycle by another rule or the current rule).

A read or write that violates its conflict condition makes the action *fail*; since rules are atomic, a failing rule is canceled in full. A *schedule* s is interpreted by running its rules in order, each from the empty rule log: a successful rule has its log appended to the schedule log L , while a failing rule leaves L unchanged. The cycle interpreter then commits L to the registers, producing their values for the next cycle.

4.2 Faultable semantics

A Kōika rule is recursively evaluated by traversing its action tree. Each node in that tree is an action that can be identified by a *path* from the root node (Figure 4). Given that, we define a fault as a pair $(p, b) : \text{path} \times \mathbb{N}$, where the *path* p points to the targeted action node and b is the bit of that node's value to flip; Figure 5 collects fault-modeling data types.

Intuitively, the fault model we formalize allows faults to be injected in the combinational part of the circuit, not directly on the register state. Note that one can target a register write node to affect the register state, partly achieving that effect. We might also consider directly faulting registers in future work.

To formalize this intuition, we introduce a new action judgment: $\mathcal{F}; \Gamma \vdash (l, a) \Downarrow_L^\sigma (l', \varphi_{\mathcal{F}}(v), \Gamma')$. $\mathcal{F}$ is a list of faults, and is threaded through the action tree: each sub-action a_i is evaluated under the routed faults $\mathcal{F} \downarrow i \triangleq \{(p, b) \mid (i :: p, b) \in \mathcal{F}\}$.

Action. As the tree is traversed, the value v produced by a node is corrupted by

$$\varphi_{\mathcal{F}} = \text{apply_fault}(f_k) \circ \dots \circ \text{apply_fault}(f_1),$$

the composition over the faults $\text{here}(\mathcal{F}) = \{f_1, \dots, f_k\}$ that target the current node, each $\text{apply_fault}(f_i)$ flipping the bit described by f_i . Note that $\varphi_{\emptyset} = \text{id}$. The semantics of faultable actions is given in the right column of Figure 6.

Rule. A rule is merely an action, run from an initial empty variable environment Γ and rule log l . We model faults at this level as a single list $\mathcal{F}$ of all faults that target the action tree of the rule (Figure 5).

Scheduler and Cycle. A schedule s is a sequence of rules, executed once per cycle. The encoding of the fault model at this level is a function from rule names to fault lists, $\mathbf{F} = \text{rule_name_}t \rightarrow \mathcal{F}$ (*cycle_fault* in Fig. 5). The schedule interpreter $\text{interp_sched_fault}$ runs the rules of s in the order prescribed by the schedule, applying for each rule its associated list of faults, and accumulating a schedule log. The cycle interpreter then commits that log to the registers σ .

$$\text{interp_cycle_fault}(s, \sigma, \mathbf{F}) \triangleq \text{commit } \sigma \ (\text{interp_sched_fault}(s, \sigma, \mathbf{F}))$$

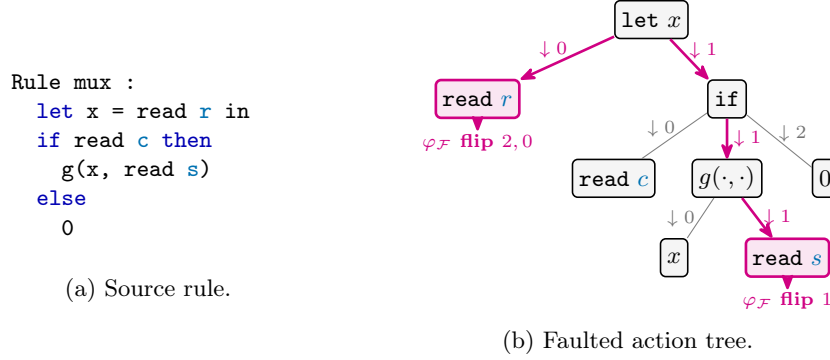


Fig. 4: A Kôika rule (a) and the fault model $\mathcal{F} = \{ ([0], 2), ([0], 0), ([1, 1, 1], 1) \}$ routed through its action tree (b). Two faults follow $\downarrow 0$ to the let-bound `read r`, where they meet ($\text{here}(\mathcal{F})$) and $\varphi_{\mathcal{F}}$ flips bits 2 and 0; the third follows $\downarrow 1$ into the body and on to `read s`, where $\varphi_{\mathcal{F}}$ flips bit 1.

$\text{path} \triangleq \text{list } \mathbb{N}$	route from the action root to a node
$\text{fault} \triangleq \text{path} \times \mathbb{N}$	a target node and the bit to flip
$\mathcal{F} : \text{list fault}$	a rule's fault model
$\text{cycle_fault} \triangleq \text{rule_name_}t \rightarrow \text{list fault}$	one fault list per rule ($\mathbf{F}$)
$\text{fault_attack} \triangleq \text{list cycle_fault}$	one cycle_fault per cycle (A)

Fig. 5: Fault-modeling data types.

Execution. A *fault attack* (Figure 5) lifts the fault modeling from a single cycle to a multi-cycle execution: an attack $A = [\mathbf{F}_1, \dots, \mathbf{F}_N]$, where each $\mathbf{F}_i$ is the cycle_fault for cycle i . Starting from the initial state σ_0 , each cycle produces the next register state $\sigma_i = \text{interp_cycle_fault}(s, \sigma_{i-1}, \mathbf{F}_i)$ for $i = 1, \dots, N$, so that $\text{interp_exec_fault}(s, \sigma_0, A) = (\sigma_0, \dots, \sigma_N)$ is the resulting trace of the register states.

This faultable semantics faithfully captures single bit-flip fault injections and supports reasoning about countermeasures. However, it is not directly amenable to SMT-assisted verification. We introduce a second modeling approach using so-called saboteurs, pieces of circuitry to directly weave faults into the circuit.

4.3 Modeling by saboteurs

Fault modeling by saboteurs rests on the same principle as the faultable semantics above: a fault corrupts the value an action produces. The only difference is that here we keep the *initial* Kôika semantics, but introduce *saboteurs* directly into the program through a compiler pass (sabotage_action). Given a *fault* to inject at bit b of the value produced by action a at cycle c , a transformation pass wraps an action a in this way:

$$\text{sabotage_action} : a \mapsto a \oplus (\text{if shouldFault}(c, \text{site_id}) \text{ then } 1 \ll b \text{ else } 0)$$

where c is read from a dedicated *cycle counter* register, site_id is a unique identifier assigned to each fault site in the program, and b is the bit to flip; shouldFault then decides whether a fault fires at that cycle and site. The $\oplus$ symbol is a XOR operation, flipping bit number b when appropriate, and leaving

	Initial	Faultable
CST	$\frac{v \in val}{\Gamma \vdash (l, v) \Downarrow (l, v, \Gamma)}$	$\frac{v \in val}{\mathcal{F}; \Gamma \vdash (l, v) \Downarrow (l, \varphi_{\mathcal{F}}(v), \Gamma)}$
VAR	$\frac{\Gamma(x) = v}{\Gamma \vdash (l, x) \Downarrow (l, v, \Gamma)}$	$\frac{\Gamma(x) = v}{\mathcal{F}; \Gamma \vdash (l, x) \Downarrow (l, \varphi_{\mathcal{F}}(v), \Gamma)}$
READ	$\frac{\text{may_read}(L, r)}{\Gamma \vdash (l, \text{read } r) \Downarrow (l \uparrow \uparrow [\text{rd}(r)], \sigma(r), \Gamma)}$	$\frac{\text{may_read}(L, r)}{\mathcal{F}; \Gamma \vdash (l, \text{read } r) \Downarrow (l \uparrow \uparrow [\text{rd}(r)], \varphi_{\mathcal{F}}(\sigma(r)), \Gamma)}$
ASSIGN	$\frac{\Gamma \vdash (l, a) \Downarrow (l', v, \Gamma')}{\Gamma \vdash (l, x := a) \Downarrow (l', \text{tt}, \Gamma'[x \mapsto v])}$	$\frac{\mathcal{F}\downarrow 0; \Gamma \vdash (l, a) \Downarrow (l', v, \Gamma')}{\mathcal{F}; \Gamma \vdash (l, x := a) \Downarrow (l', \varphi_{\mathcal{F}}(\text{tt}), \Gamma'[x \mapsto v])}$
WRITE	$\frac{\Gamma \vdash (l, \text{write } r \ a) \Downarrow (l' \uparrow \uparrow [\text{wr}(r, v)], \text{tt}, \Gamma')}{\Gamma \vdash (l, a_1) \Downarrow (l', v_1, \Gamma')}$	$\frac{\mathcal{F}; \Gamma \vdash (l, \text{write } r \ a) \Downarrow (l' \uparrow \uparrow [\text{wr}(r, v)], \varphi_{\mathcal{F}}(\text{tt}), \Gamma')}{\mathcal{F}\downarrow 0; \Gamma \vdash (l, a_1) \Downarrow (l', v_1, \Gamma')}$
SEQ	$\frac{\Gamma \vdash (l, a_1) \Downarrow (l', v_1, \Gamma')}{\Gamma \vdash (l, a_1; a_2) \Downarrow (l'', v, \Gamma'')}$	$\frac{\mathcal{F}\downarrow 1; \Gamma' \vdash (l', a_2) \Downarrow (l'', v, \Gamma'')}{\mathcal{F}; \Gamma \vdash (l, a_1; a_2) \Downarrow (l'', \varphi_{\mathcal{F}}(v), \Gamma'')}$
BIND	$\frac{(x, v_1) :: \Gamma' \vdash (l', a_2) \Downarrow (l'', v, \Gamma'')}{\Gamma \vdash (l, \text{let } x = a_1 \text{ in } a_2) \Downarrow (l'', v, \text{tl } \Gamma'')}$	$\frac{\mathcal{F}\downarrow 1; (x, v_1) :: \Gamma' \vdash (l', a_2) \Downarrow (l'', v, \Gamma'')}{\mathcal{F}; \Gamma \vdash (l, \text{let } x = a_1 \text{ in } a_2) \Downarrow (l'', \varphi_{\mathcal{F}}(v), \text{tl } \Gamma'')}$
IF-TRUE	$\frac{\Gamma \vdash (l, \text{if } a_1 \text{ then } a_2 \text{ else } a_3) \Downarrow (l'', v, \Gamma'')}{\Gamma \vdash (l, a_1) \Downarrow (l', [\text{true}], \Gamma')}$	$\frac{\mathcal{F}; \Gamma \vdash (l, \text{if } a_1 \text{ then } a_2 \text{ else } a_3) \Downarrow (l'', \varphi_{\mathcal{F}}(v), \Gamma'')}{\mathcal{F}\downarrow 0; \Gamma \vdash (l, a_1) \Downarrow (l', [\text{true}], \Gamma')}$
IF-FALSE	$\frac{\Gamma \vdash (l, \text{if } a_1 \text{ then } a_2 \text{ else } a_3) \Downarrow (l'', v, \Gamma'')}{\Gamma \vdash (l, a_1) \Downarrow (l', [\text{false}], \Gamma')}$	$\frac{\mathcal{F}\downarrow 2; \Gamma' \vdash (l', a_3) \Downarrow (l'', v, \Gamma'')}{\mathcal{F}; \Gamma \vdash (l, \text{if } a_1 \text{ then } a_2 \text{ else } a_3) \Downarrow (l'', \varphi_{\mathcal{F}}(v), \Gamma'')}$
FUNCALL	$\frac{\text{zip}(\text{args } f, [v_1, \dots, v_n]) \vdash (l_n, \text{body } f) \Downarrow (l', v, \Gamma')}{\Gamma_0 \vdash (l_0, f(a_1, \dots, a_n)) \Downarrow (l', v, \Gamma_n)}$	$\frac{\mathcal{F}\downarrow n; \text{zip}(\text{args } f, [v_1, \dots, v_n]) \vdash (l_n, \text{body } f) \Downarrow (l', v, \Gamma')}{\mathcal{F}; \Gamma_0 \vdash (l_0, f(a_1, \dots, a_n)) \Downarrow (l', \varphi_{\mathcal{F}}(v), \Gamma_n)}$

Fig. 6: Kôika action evaluation semantics: initial (left) and faultable (right).

Rule mux :

```

let x = read r ⊕ (if shouldFault(cycle,0) then 0b100 else 0)
              ⊕ (if shouldFault(cycle,1) then 0b001 else 0) in
if read c then
  g(x, read s ⊕ (if shouldFault(cycle,2) then 0b010 else 0))
else 0

```

Fig. 7: The rule from Figure 4 sabotaged under the same fault model $\mathcal{F} = \{ ([0], 2), ([0], 0), ([1, 1, 1], 1) \}$.

the value unchanged otherwise. This is illustrated in Figure 7 under the same fault model as in Figure 4.

The external call `shouldFault(c, site_id)` compiles to a free SMT-LIB variable, so the solver chooses which faults fire: in this way, we delegate the construction of *fault attacks* (A) to the solver and automates the verification. For now we trust Z3 [8] and the SMT encoding of the circuit semantics.

4.4 Framework workflow

Each approach feeds its own proof method—deductive Rocq for the faultable semantics, automated Z3 [8] assisted for the sabotaged circuit—so a property can be discharged on whichever is convenient. This relies on their executions being equivalent under the same fault attack (Theorem 1), which is proved in Rocq.

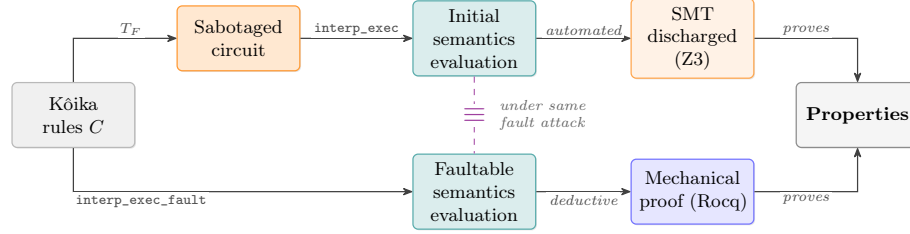


Fig. 8: Semantic and Sabotage fault modeling equivalence and proof workflow.

Theorem 1 (Saboteur equivalence). *For every schedule s , state σ , fault attack $A = [\mathbf{F}_1, \dots, \mathbf{F}_N]$, and flag function `shouldFault`,*

$$\begin{aligned}
 & (\forall c, \forall \text{site}, \text{shouldFault}(c, \text{site}) = 1 \iff \text{faultable_cycle}(\text{site}, \mathbf{F}_c)) \\
 & \implies \text{interp_exec}(\text{sabotage_sch}(s), \sigma) = \text{interp_exec_fault}(s, \sigma, A).
 \end{aligned}$$

5 Proof of k -fault-resistant partitioning

We prove the k -fault-resistant partitioning property of the 8-bit partial-product multiplier of Section 3—a Kôika circuit hardened with a CED—using our SMT-based automation.

5.1 Adversary model

We assume an adversary that can inject at most $k = 1$ bit-flips fault anywhere in the circuit except the shared *fetch* stage. Formally, a fault (p, b) flips bit b of the value produced at the node reached by path p ; an attack $\mathcal{A}$ thus targets the nodes $\text{tgt}(\mathcal{A}) \triangleq \{(r, p) : (p, b) \in \mathcal{F}_r, \mathcal{F}_r \in \mathbf{F}, \mathbf{F} \in \mathcal{A}\}$, where r ranges over the rules of the schedule. Let $\mathcal{S}$ collect the nodes of the shared *fetch* rule, which reads the operands and feeds both pipelines. Our adversary is any attack $\mathcal{A}$ with $\sum_{\mathbf{F} \in \mathcal{A}} \sum_{\mathcal{F} \in \mathbf{F}} |\mathcal{F}| \leq k = 1$ and $\text{tgt}(\mathcal{A}) \cap \mathcal{S} = \emptyset$: at most one bit-flip, none in the rule *fetch* which is outside the domain of the countermeasure.

5.2 Proof obligation

As noted in Section 3, k -fault-resistant partitioning unrolls $l = \max(1, d)$ transitions; here the CED is immediate ($d = 0$), so $l = 1$. We run that single cycle on two same-input executions: a fault-free *golden* run and a possibly faulty one under an attack of at most one fault.

A state $\sigma: \mathcal{R} \rightarrow \text{val}$ assigns values to the registers in $\mathcal{R}$. Both runs use the schedule s and solver-chosen inputs (A_γ, B_γ) golden, (A_σ, B_σ) faulty). The golden run is the fault-free $(\gamma_0, \gamma_1) \triangleq \text{interp_exec_golden}(s, \gamma_0)$ and the faulty run $(\sigma_0, \sigma_1) \triangleq \text{interp_exec_fault}(s, \sigma_0, \mathcal{A})$ under attack $\mathcal{A} = [\mathbf{F}_1]$; here σ_0, σ_1 (resp. γ_0, γ_1) are the faulty (resp. golden) initial and next states. We *fix* this following partitioning by hand, rather than searching for a suitable one as Tollec et al. [15] do: $\mathcal{P} \triangleq \{\pi_1, \pi_2, \pi_3\}$ where

$\pi_i = \{\text{fromFetch_}i, \text{fromPartial_}i, \text{output_}i\}$ for $i \in \{1, 2\}$ and $\pi_3 = \{\text{alert}\}$. Observe that a duplicated region and its replica always lie in separate

partitions. We define the function $\text{mir}: \pi_1 \xrightarrow{\sim} \pi_2$ which maps each register of the first partition to its counterpart in the second. We set

$$\begin{aligned} \sigma_j \approx_\pi \gamma_j &\triangleq \forall r \in \pi, \sigma_j(r) = \gamma_j(r) && \text{the runs agree on } \pi \text{ at cycle } j \\ \text{diff}(j) &\triangleq |\{\pi \in \mathcal{P} : \sigma_j \not\approx_\pi \gamma_j\}| && \text{partitions differing at cycle } j \\ \text{faults}(j) &\triangleq \sum_{\mathcal{F} \in \mathbf{F}_j} |\mathcal{F}| && \text{bit-flips injected in cycle } j \\ \|\mathcal{A}\| &\triangleq \sum_{j=1}^{|\mathcal{A}|} \text{faults}(j) && \text{fault budget: total bit-flips of } \mathcal{A} \end{aligned}$$

We now state Theorem 2, which we prove using Z3 [8] on the 8-bit partial-product multiplier in about 15 minutes using less than 1GB of RAM, on a regular laptop (Intel(R) Core(TM) Ultra 7 265H with 32GB of RAM).

Theorem 2 (one_fault_resistant_partitioning). *For all operands $A_\gamma, B_\gamma, A_\sigma, B_\sigma$ feeding the two runs the same inputs, and every attack $\mathcal{A} = [\mathbf{F}_1]$ with $\|\mathcal{A}\| \leq 1$, the golden trace $(\gamma_0, \gamma_1) \triangleq \text{interp_exec_golden}(s, \gamma_0)$ and the faulty trace $(\sigma_0, \sigma_1) \triangleq \text{interp_exec_fault}(s, \sigma_0, \mathcal{A})$ satisfy:*

$$\begin{aligned} &[\forall j \in \{0, 1\}, \sigma_j(\text{alert}) = 0] \wedge [\forall r \in \pi_1, \gamma_0(r) = \gamma_0(\text{mir}(r))] \wedge \\ &[\text{diff}(0) \leq 1] \wedge [\text{diff}(0) + \text{faults}(1) \leq 1] \\ \implies &[\text{diff}(1) \leq \text{diff}(0) + \text{faults}(1)] \wedge [\sigma_1(\text{output_1}) = \gamma_1(\text{output_1})]. \end{aligned}$$

The two conjuncts in the conclusion are respectively the *fault confinement* and *output integrity* obligations.

Scaling with design size. The circuit size increases the number of fault sites. Each fault site is associated with a boolean variable controlled by the solver. Complexity grows linearly with the number of fault sites, and exponentially with the state size. However, this does not directly correlates with the solver's execution time, since solvers employ many heuristics that may prune entire parts of the search space.

6 Conclusion and Future Work

We took a first step toward formalizing fault injection in Kôika: a *faultable* semantics for deductive reasoning in Rocq, and a *sabotage-based* transformation pass amenable to SMT-assisted verification. We proved the equivalence of the two approaches. We also automatically discharged the *k-fault-resistant partitioning* property of a duplicated multiplier using Z3 [8]. As future work, we plan to verify larger circuits, such as CPUs. More ambitiously, we aim for *fault security by construction* of Kôika circuits, leveraging the formal semantics and the fact that some aspects of properties such as *k-fault-resistant partitioning* are *structural* rather than circuit-specific.

Acknowledgments. This work benefited from French State support as part of high-risk research program Audace!, led by the CEA and managed by the Agence Nationale de la Recherche under the France 2030 heading, bearing the reference ANR-24-RR11-0004.

References

1. Arribas, V., Wegener, F., Moradi, A., Nikova, S.: Cryptographic Fault Diagnosis using VerFI. In: 2020 IEEE International Symposium on Hardware Oriented Security and Trust (HOST) (Dec 2020)

2. Baty, M., Wilke, P., Hiet, G., Fontaine, A., Trieu, A.: A generic framework to develop and verify security mechanisms at the microarchitectural level: Application to control-flow integrity. In: 36th IEEE Computer Security Foundations Symposium, CSF 2023, Dubrovnik, Croatia, July 10-14, 2023. pp. 372–387. IEEE (2023)
3. Bourgeat, T., Pit-Claudel, C., Chlipala, A., Arvind: The essence of bluespec: a core language for rule-based hardware design. In: Donaldson, A.F., Torlak, E. (eds.) Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2020, London, UK, June 15-20, 2020. pp. 243–257. ACM (2020)
4. Burchard, J., Gay, M., Ekossono, A.S.M., Horáček, J., Becker, B., Schubert, T., Kreuzer, M., Polian, I.: AutoFault: Towards Automatic Construction of Algebraic Fault Attacks. In: 2017 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC) (2017)
5. Ducouso, S., Bardin, S., Potet, M.L.: Adversarial Reachability for Program-level Security Analysis. In: 32nd European Symposium on Programming (ESOP). pp. 59–89 (2023)
6. Hoffmann, M., Schellenberg, F., Paar, C.: ARMORY: Fully Automated and Exhaustive Fault Simulation on ARM-M Binaries. IEEE Transactions on Information Forensics and Security **16** (2021)
7. Laurent, J., Deleuze, C., Pebay-Peyroula, F., Beroulle, V.: Bridging the Gap between RTL and Software Fault Injection. ACM Journal on Emerging Technologies in Computing Systems **17**(3) (May 2021)
8. de Moura, L., Bjørner, N.: Z3: An efficient smt solver. In: Ramakrishnan, C.R., Rehof, J. (eds.) Tools and Algorithms for the Construction and Analysis of Systems. pp. 337–340. Springer Berlin Heidelberg, Berlin, Heidelberg (2008)
9. Nguyen, V.S., Grosso, V., Cayrel, P.L.: Practical persistent fault attacks on AES with instruction skip. IACR Communications in Cryptology **2**(1) (2025)
10. Nikhil, R.: Bluespec system verilog: efficient, correct rtl from high level specifications. In: Proceedings. Second ACM and IEEE International Conference on Formal Methods and Models for Co-Design, 2004. MEMOCODE '04. pp. 69–70 (2004)
11. Potet, M.L., Mounier, L., Puys, M., Dureuil, L.: Lazart: A Symbolic Approach for Evaluation the Robustness of Secured Codes against Control Flow Injections. In: Verification and Validation 2014 IEEE Seventh International Conference on Software Testing (Mar 2014)
12. Richter-Brockmann, J., Rezaei Shahmirzadi, A., Sasdrich, P., Moradi, A., Güneysu, T.: FIVER – Robust Verification of Countermeasures against Fault Injections. IACR Transactions on Cryptographic Hardware and Embedded Systems (Aug 2021)
13. Richter-Brockmann, J., Sasdrich, P., Bache, F., Güneysu, T.: Concurrent error detection revisited: hardware protection against fault and side-channel attacks. In: Proceedings of the 15th International Conference on Availability, Reliability and Security. ARES '20, Association for Computing Machinery, New York, NY, USA (2020)
14. Tollec, S., Asavae, M., Couroussé, D., Heydemann, K., Jan, M.: μ archifi: Formal modeling and verification strategies for microarchitectural fault injections. In: FMCAD (2023)
15. Tollec, S., Hadzic, V., Nasahl, P., Asavae, M., Bloem, R., Couroussé, D., Heydemann, K., Jan, M., Mangard, S.: Fault-resistant partitioning of secure cpus for system co-verification against faults. TCHES (2024)